

EC340: Computer Organization and Architecture

Processor Design

1

Lecture Plan

- Introduction
 - building blocks
- A simple implementation: Single Cycle
 - Data path and control
- Performance considerations
- Multi-cycle design
 - Data path and control
- Exception handling

2

Outline

- MIPS subset for implementation
- Design overview
- Division into data path and control
- Building blocks - combinational and sequential
- Clock and timings
- Components required for MIPS subset

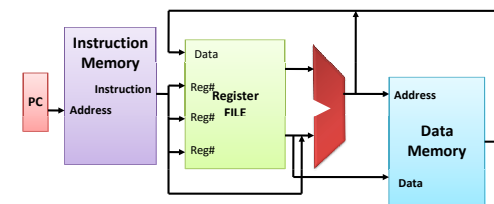
MIPS subset for implementation

- Arithmetic - logic instructions
 - **add, sub, and, or, slt**
- Memory reference instructions
 - **lw, sw**
- Control flow instructions
 - **beq, j**

Generic Implementation

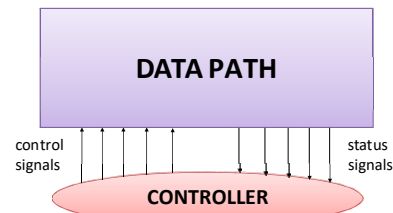
- Use the program counter (PC) to supply instruction address
- Get the instruction from memory
- Read registers
- Use the instruction to decide exactly what to do

Design overview



6

Division into Data path and Control



Building block types

Two types of functional units:

- Elements that operate on data values (combinational)
 - Output is function of current input
 - No memory
- Elements that contain state (sequential)
 - Output is function of current and previous inputs
 - State = memory

Combinational circuit examples

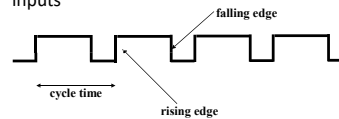
- Gates: and, or, nand, nor, xor, inverter
- Multiplexer
- Decoder
- Adder, subtractor, comparator
- ALU
- Array multipliers

Sequential circuit examples

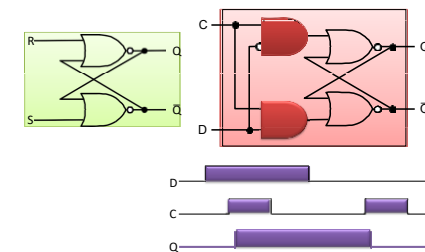
- Flip-flops
- Counters
- Registers
- Memories

Clocked vs. unclocked circuit

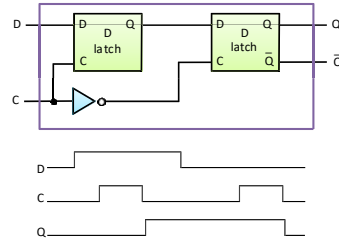
- Clocked state element
 - State changes only with clock edge
- Unclocked state element
 - State changes can occur with changes in other inputs



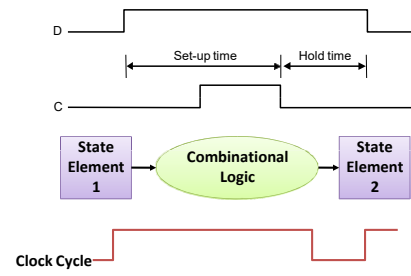
Unclocked state elements



Clocked State Elements



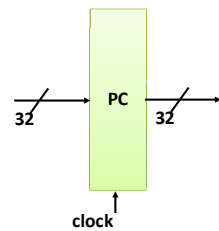
Clock and timings



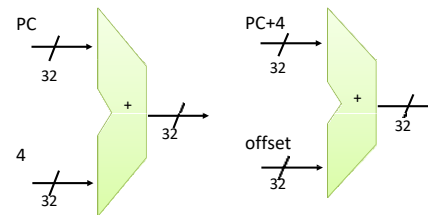
Components for MIPS subset

- Register
- Adder
- ALU
- Multiplexer
- Register file
- Program memory
- Data memory
- Bit manipulation components

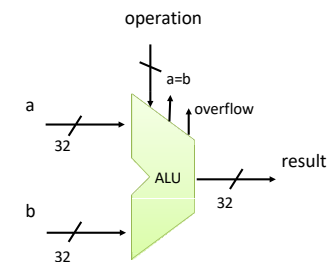
MIPS Components - Register



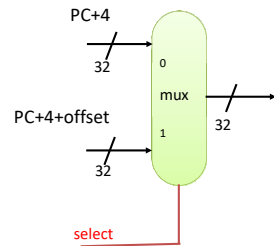
MIPS Components - Adder



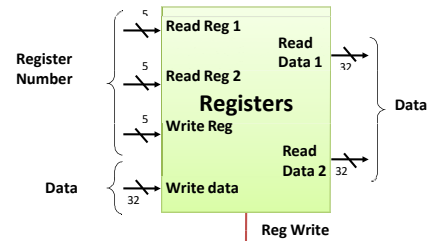
MIPS Components - ALU



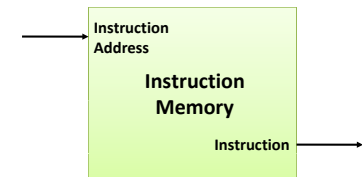
MIPS components - Multiplexers



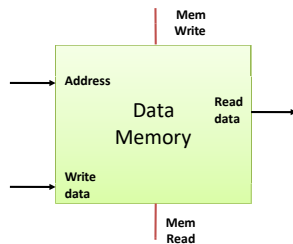
MIPS Components - register file



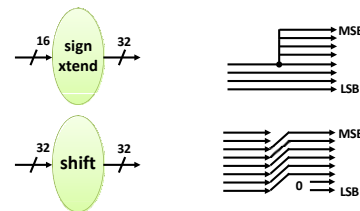
MIPS Components: Program memory



MIPS Components -Data memory



MIPS Components - Bit manipulation circuits



Summary

- MIPS subset chosen – 9 instructions
- Design organized as data path + control
- Building blocks – combinational and sequential, clocked and unclocked
- Clock and timings
- Components for MIPS subset – register, mux, adder, ALU, register file, program and data memories

A Processor Design Method

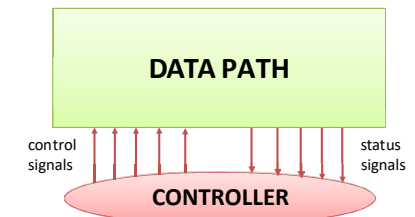
Build the datapath step by step as follows

- Start with R - class instructions
- Include other instructions one by one
- Identify control signals
- Interconnect datapath and controller

MIPS subset for implementation

- Arithmetic - logic instructions
 - add, sub, and, or, slt
- Memory reference instructions
 - lw, sw
- Control flow instructions
 - beq, j

Division into data path and control



Datapath for add,sub,and,or,slt

- fetch instruction
 - address the register file
 - pass operands to ALU
 - pass result to register file
 - increment PC
- } actions required

Format: add \$t0, \$s1, \$s2

000000	10001	10010	01000	00000	100000
op	rs	rt	rd	shamt	funct

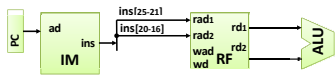
Fetching instruction



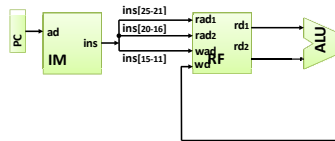
Addressing RF



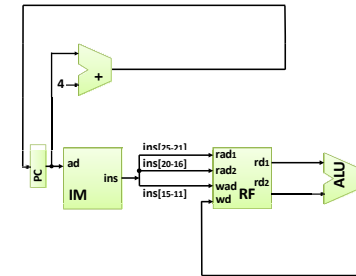
Passing operands to ALU



Passing the result to RF



Incrementing PC

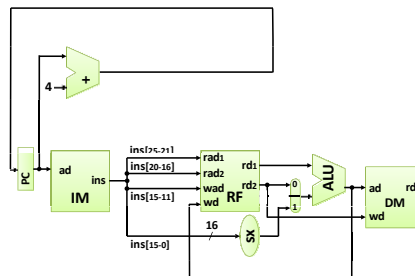


Load and Store instructions

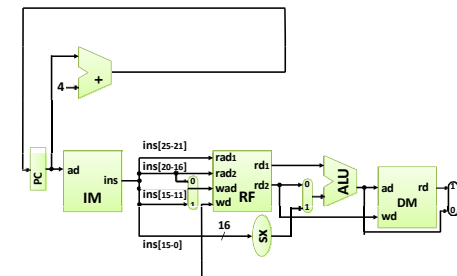
- format : I
- Example: lw \$t0, 32(\$s2)

35	18	9	32
op	rs	rt	16 bit number

Adding "sw" instruction



Adding "lw" instruction

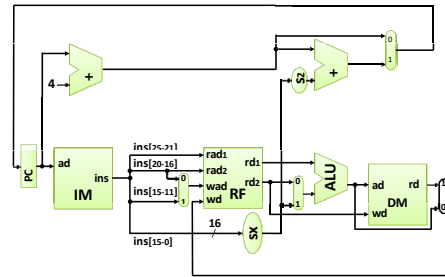


Format of beq instruction

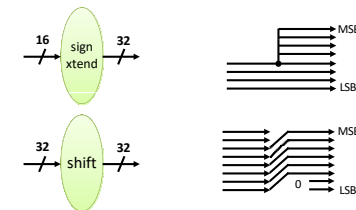
- beq I - format

op	rs	rt	16 bit number
----	----	----	---------------

Adding "beq" instruction



MIPS components - bit manipulation circuits

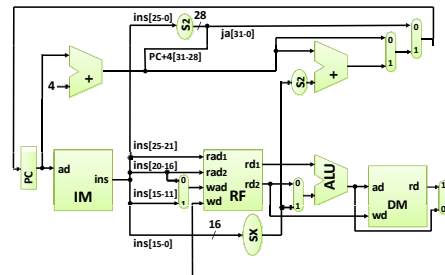


Format of jump instruction

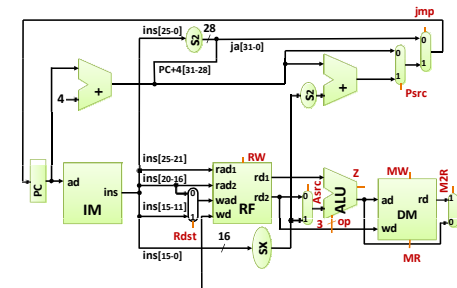
- j J - format

op	26 bit number
----	---------------

Adding "j" instruction



Control signals



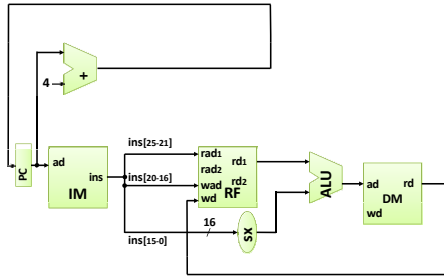
- Processor designed for {add, sub, and, or, slt, lw, sw, beq, j}
- Step by step approach
- Started with {add, sub, and, or, slt}
- Added {sw, lw}, then added {beq, j}
- Identified control signals and connected to a controller (black box).

- Build the datapath by merging designs for individual instructions
- Define control inputs for various instructions
- Build truth table for controller
- Analyzing delays
- Problems with single cycle datapath

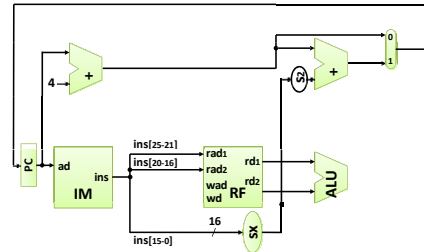
- Grow the design incrementally
 - D1: {add, sub, and, or, slt}
 - D2: {add, sub, and, or, slt, sw}
 - D3: {add, sub, and, or, slt, sw, lw}
 - D4: {add, sub, and, or, slt, sw, lw, beq}
 - D5: {add, sub, and, or, slt, sw, lw, beq, j}
- Do sub-designs and then merge
 - D1: {add, sub, and, or, slt}
 - D2: {sw}
 - D3: {lw}
 - D4: {beq}
 - D5: {j}
 - D6: $D1 \cup D2 \cup D3 \cup D4 \cup D5$

The diagram illustrates the MIPS processor architecture. The PC (Program Counter) feeds into the Instruction Memory (IM) and the ALU. The IM outputs instructions to the ALU. The ALU outputs results to the Register File (RF). The RF outputs results to the IM and the ALU. The ALU also outputs results to the PC via a 4-bit adder.

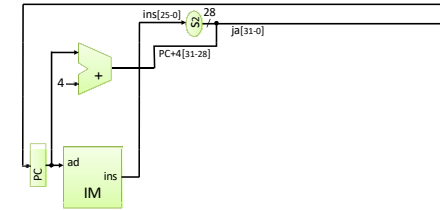
Design for {lw}



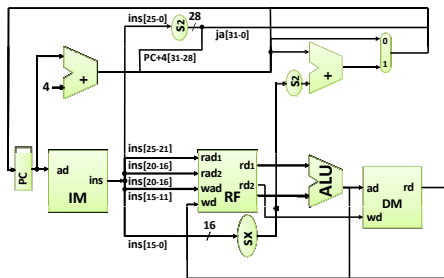
Design for {beq}



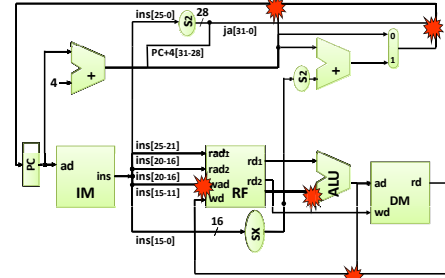
Design for {j}



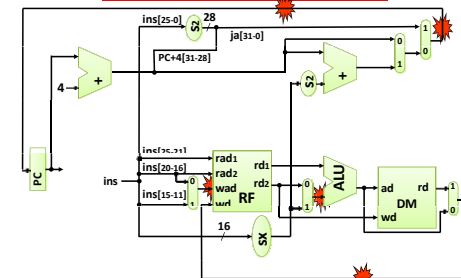
Merging sub-designs



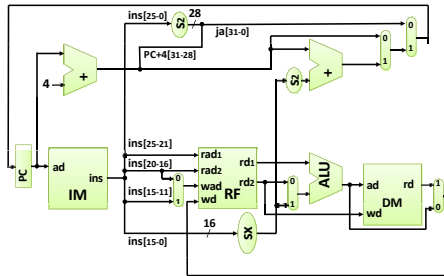
Identify conflicts



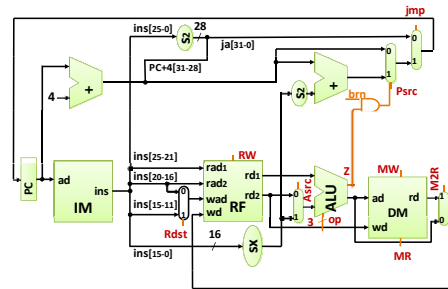
Introduce multiplexers



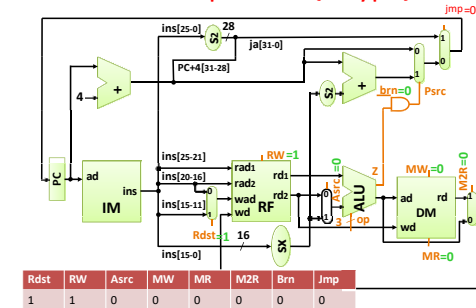
Introduce multiplexers



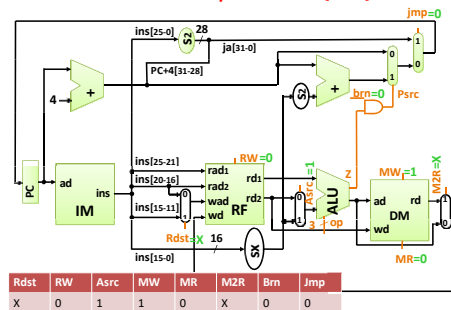
Control signals



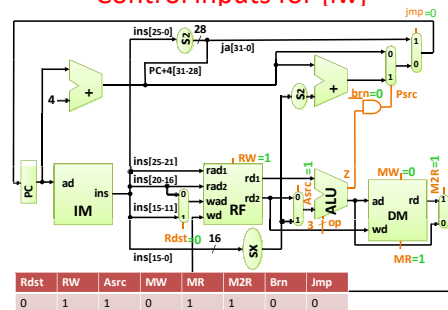
Control inputs for {R-type}



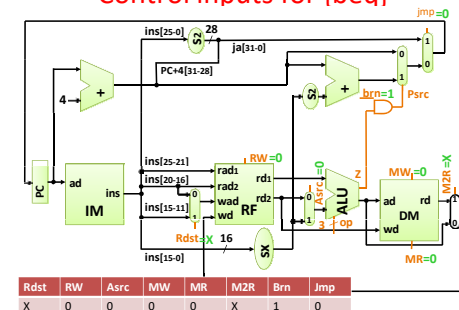
Control inputs for {sw}



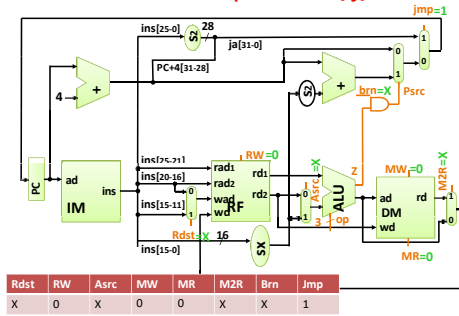
Control inputs for {lw}



Control inputs for {beq}



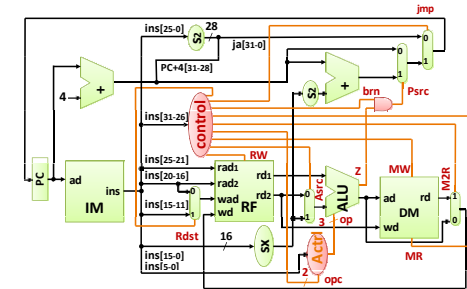
Control inputs for {j}



All control inputs

Instruction	Opcode	Rdst	RW	Asrc	MW	MR	M2R	Brn	Jmp
Rtype	000000	1	1	0	0	0	0	0	0
Sw	101011	X	0	1	1	0	X	0	0
Lw	100011	0	1	1	0	1	1	0	0
Beq	000100	X	0	0	0	0	X	1	0
J	000010	X	0	X	0	0	X	X	1

Datapath + Control



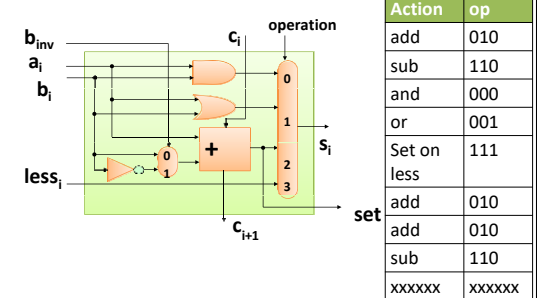
Encoding opc

Ins	Opc ode	OPC	Rdst	RW	Asrc	MW	MR	M2R	Brn	Jmp
R	000000	10	1	1	0	0	0	0	0	0
Sw	101011	00	X	0	1	1	0	X	0	0
Lw	100011	00	0	1	1	0	1	1	0	0
Beq	000100	01	X	0	0	0	0	X	1	0
J	000010	XX	X	0	X	0	0	X	X	1

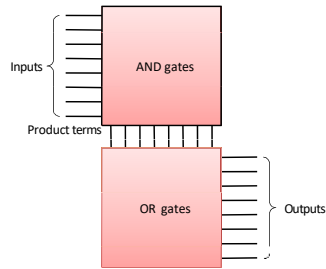
ALU control input

Type	OPC	INS	FUN	Action	OP
R-Type	10	Add	100000	Add	010
R-Type	10	Sub	100010	Sub	110
R-Type	10	And	100100	And	000
R-Type	10	Or	100101	Or	001
R-Type	10	SLT	101010	SetOnLess	111
Sw	00	Sw	Xxxxxx	Add	010
Lw	00	Lw	Xxxxxx	Add	010
Beq	01	Beq	Xxxxxx	Sub	110
J	XX	J	Xxxxxx	Xxx	xxx

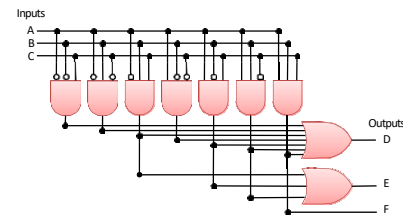
ALU Control unit



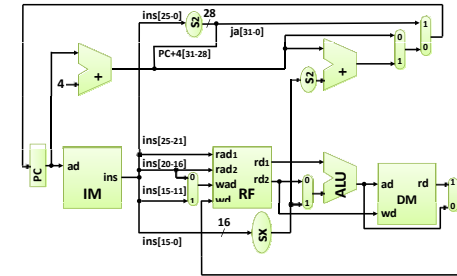
Implementing controllers



AND-OR PLA



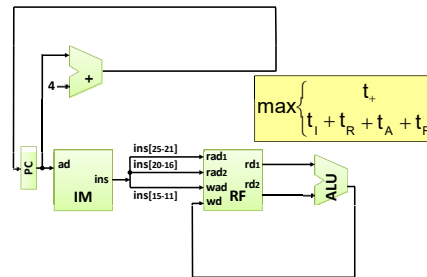
Single Cycle Datapath



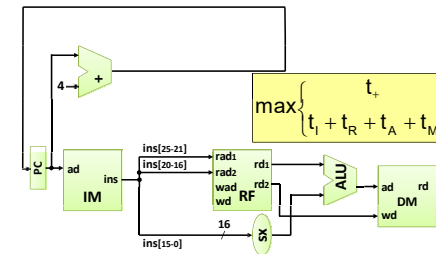
Analyzing performance

Components	Delay
Register	0
Adder	t_+
ALU	t_A
MUX	0
RF	t_R
Instruction Memory	t_I
Data Memory	t_M
Bit manipulation	0

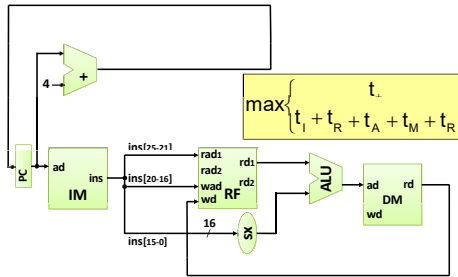
Delay for {add, sub, and, or, slt}



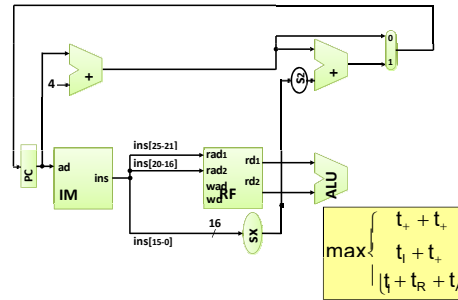
Delay for {sw}



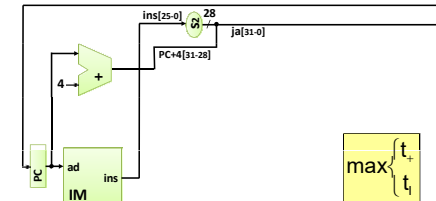
Delay for {lw}



Delay for {beq}



Delay for {j}



Overall clock period

$$\max \begin{cases} R : t_+, t_I + t_R + t_A + t_R \\ SW : t_+, t_I + t_R + t_A + t_M \\ LW : t_+, t_I + t_R + t_A + t_M + t_R \\ Beq : t_+, t_+, t_I + t_+, t_I + t_R + t_A \\ J : t_+, t_I \end{cases}$$

or

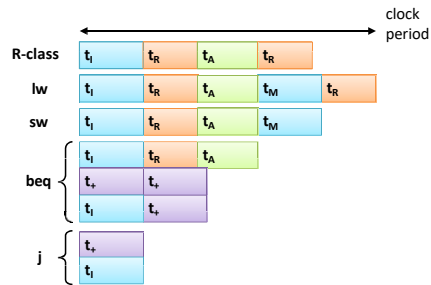
$$\max \begin{cases} t_I + t_R + t_A + t_M + t_R \\ t_+ + t_+ \\ t_I + t_+ \end{cases}$$

Problems with single cycle design

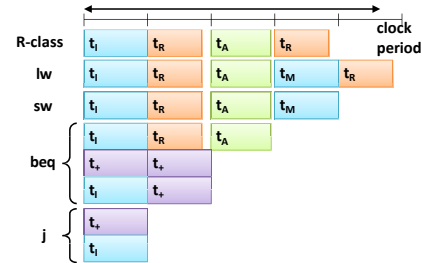
- Slowest instruction pulls down the clock frequency
- Resource utilization is poor
- There are some instructions which are impossible to be implemented in this manner

Multi-Cycle Design

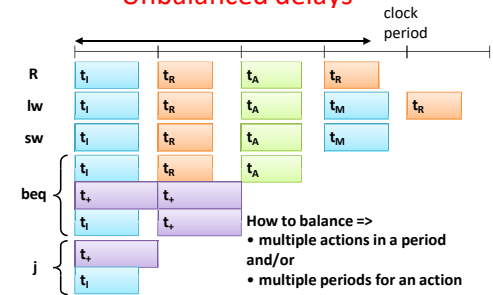
Clock period in single cycle design



Clock period in multi-cycle design



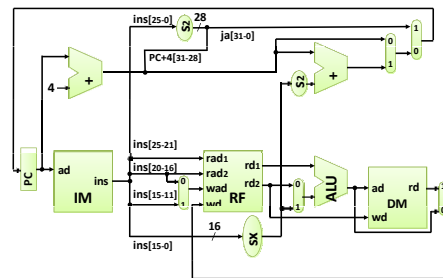
Unbalanced delays



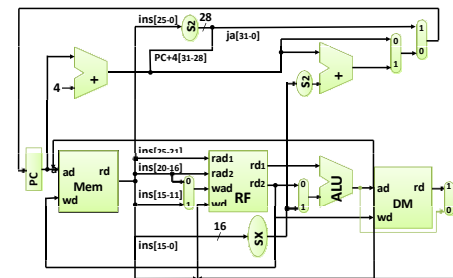
Improving resource utilization

- Can we eliminate two adders?
- How to share (or reuse) a resource (say ALU) in different clock cycles?
- Store results in registers.
- more multiplexing may be required!
- Resources in this design: RF, ALU, MEM.

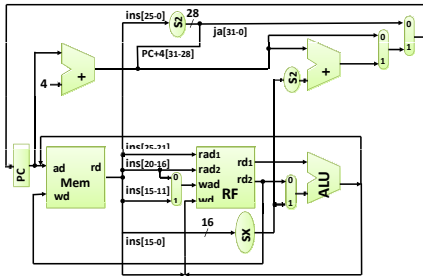
Single Cycle Datapath



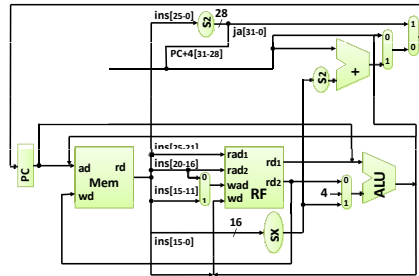
Merge IM and DM



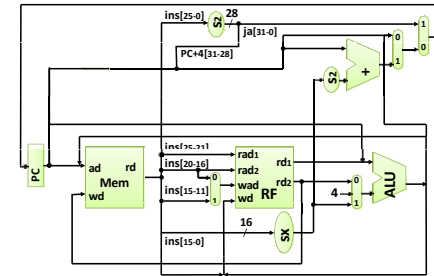
Rearrange diagram



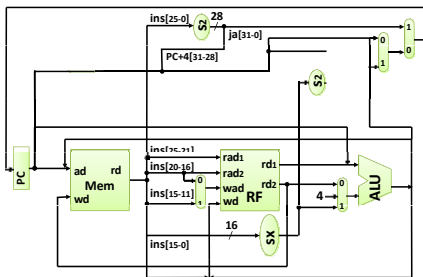
Eliminate first adder



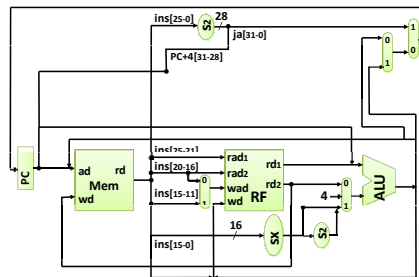
Eliminate first adder



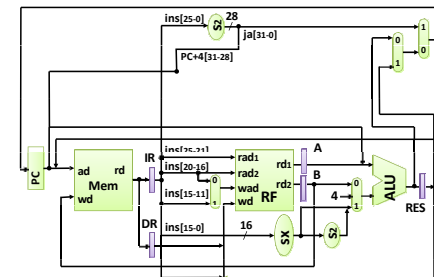
Eliminate second adder



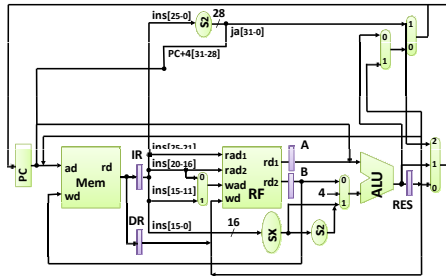
Eliminate second adder



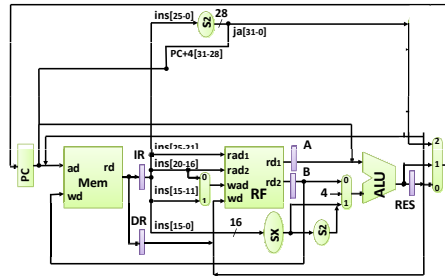
Introduce registers



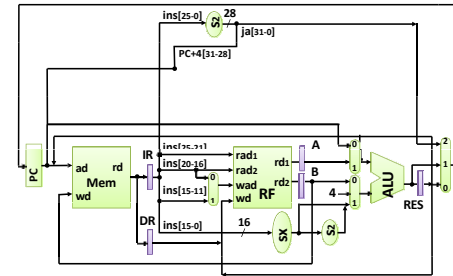
Rearrange PC input multiplexer



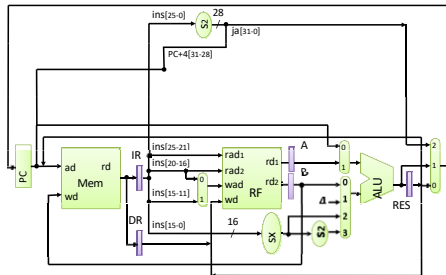
Rearrange PC input multiplexer



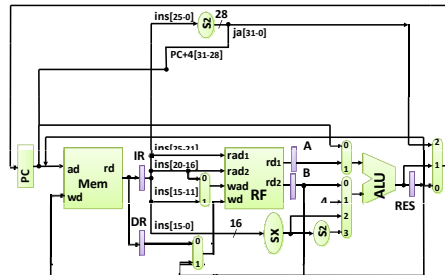
Introduce ALU inp1 multiplexer



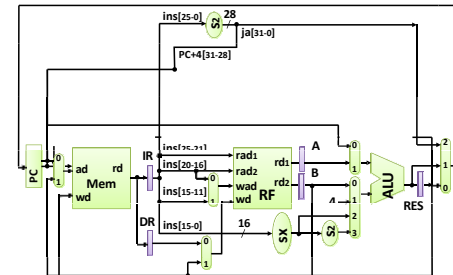
Rearrange ALU inp2 multiplexer



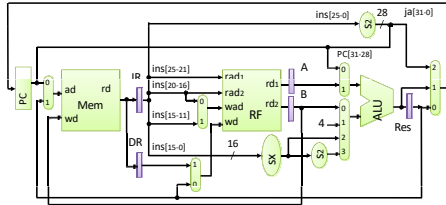
Introduce RF inp multiplexer



Introduce Mem inp multiplexer



Final: Multi-Cycle Data Path



22

Multi-cycle data path

- Single cycle approach to multi-cycle approach: improve performance and resource sharing
- Delays in different cycles should be balanced
- Single ALU and single memory used
- Additional registers and multiplexers required

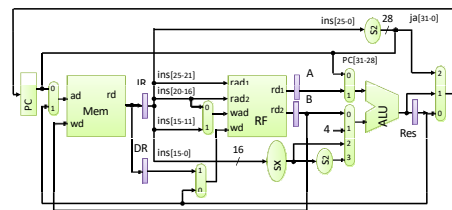
Multi-cycle Control

24

Multi-cycle Control Design

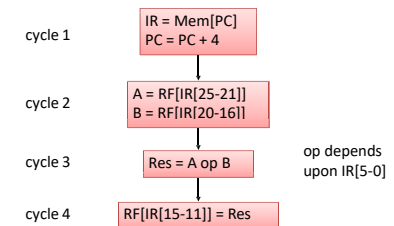
- Break instructions into cycles
- Put cycle sequences together
- Control signal groups and micro operations
- Control states and signal values
- Control state transitions

Multi-Cycle Data Path

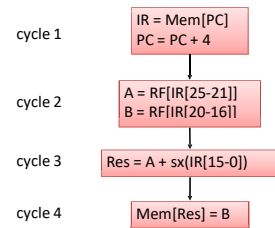


26

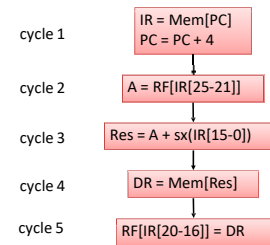
Break Instruction Execution into Cycles: R-class instructions



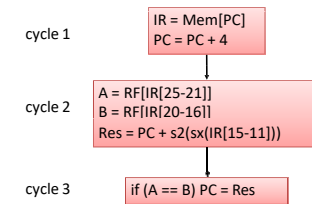
Break Instruction Execution into Cycles: sw instruction



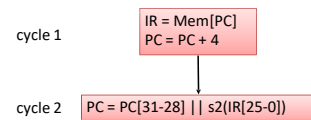
Break Instruction Execution into Cycles: lw instruction



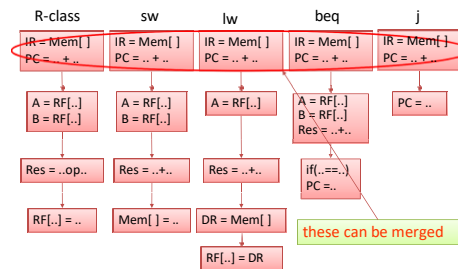
Break Instruction Execution into Cycles: beq instruction



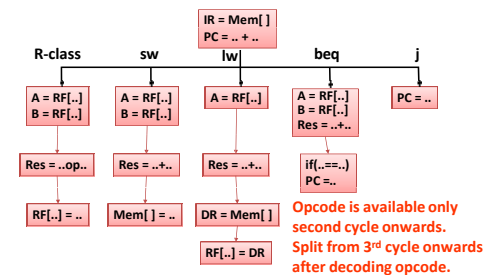
Break Instruction Execution into Cycles: j instruction



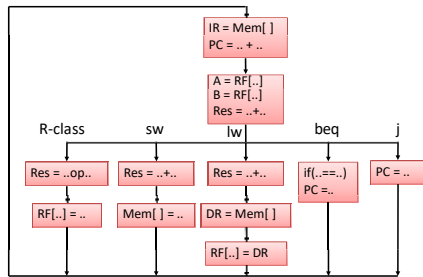
Put cycle sequences together



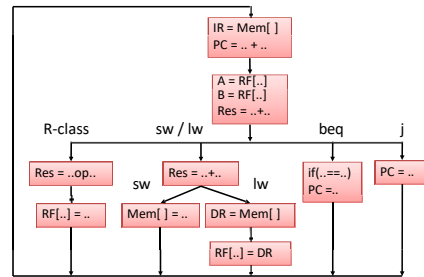
After merging fetch cycle



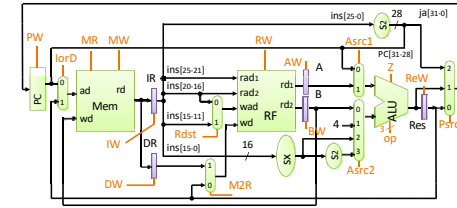
With a common decoding cycle



lw, sw can split after third cycle



Control signals in multi-cycle DP



Micro operations and control signals - PC group

Micro operation		PWu	PWc	Psrc
PC = PC + 4	PCinc	1	X	1
if (A == B) PC = Res	branch	0	1	0
PC=PC[31-28] s2(IR[25-0])	jump	1	X	2
default	nop	0	0	X

$$PW = PWu + Z \cdot PWc$$

Micro operations and control signals - Mem group

Micro operation		MW	MR	IorD	IW	DW
IR = Mem[PC]	fetch	0	1	0	1	0
DR = Mem[Res]	m_rd	0	1	1	0	1
Mem[Res] = R	m_wr	1	0	1	0	0
default	nop	0	0	X	0	0

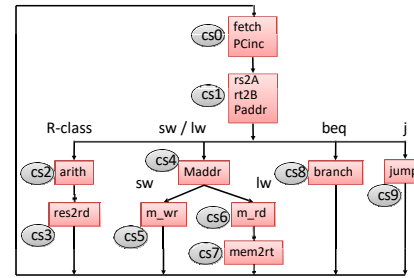
Micro operations and control signals - RF group

Micro operation		RW	Rdst	M2R	AW	BW
A = RF[IR[25-21]]	rs2A	0	X	X	1	0
B = RF[IR[20-16]]	rt2B	1	X	X	0	1
RF[IR[15-11]] = Res	res2rd	2	1	0	0	0
RF[IR[20-16]] = DR	mem2rt	1	0	1	0	0
default	nop	0	X	X	0	0

Micro operations and control signals - ALU group

Micro operation		opc	Asrc1	Asrc2	ReW
PC = PC + 4	PCinc	0	0	1	0
Res = A op B	arith	2	1	0	1
Res = A + sx(IR[15-0])	Maddr	0	1	2	1
Res = PC + s2(sx(IR[15-11]))	Paddr	0	0	3	1
if (A == B) PC = Res	branch	1	1	0	0
default	nop	X	X	X	0

Control states and micro operations



Control states and signal values

	PC grp	Mem grp	RF grp	ALU grp
cs0	PCinc	fetch	nop	PCinc
cs1	nop	nop	rs2A,rt2B	Paddr
cs2	nop	nop	nop	arith
cs3	nop	nop	res2rd	nop
cs4	nop	nop	nop	Maddr
cs5	nop	m_wr	nop	nop
cs6	nop	m_rd	nop	nop
cs7	nop	nop	mem2rt	nop
cs8	branch	nop	nop	branch
cs9	jump	nop	nop	nop

Control state transitions

	R-class	sw	lw	beq	j
cs0	cs1	cs1	cs1	cs1	cs1
cs1	cs2	cs4	cs4	cs8	cs9
cs2	cs3	X	X	X	X
cs3	cs0	X	X	X	X
cs4	X	cs5	cs6	X	X
cs5	X	cs0	X	X	X
cs6	X	X	cs7	X	X
cs7	X	X	cs0	X	X
cs8	X	X	X	cs0	X
cs9	X	X	X	X	cs0

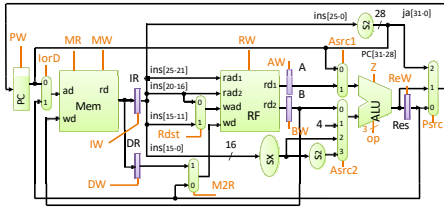
Summary

- Instructions expressed as sequences of micro operations
- Control signals are grouped
- Micro operations define values of control signals of a group
- Control states associated with micro operations
- Control state transitions depend upon opcode

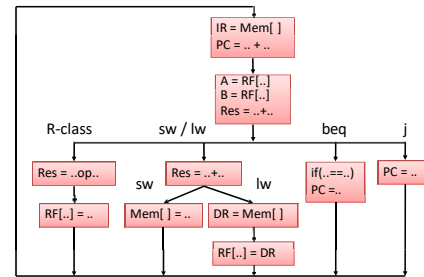
Lecture Plan

- Introduction
 - building blocks
- A simple implementation: Single Cycle
 - Data path and control
- Performance considerations
- Multi-cycle design
 - Data path and control
- Exception handling

Control signals in multi-cycle DP



Control state transition diagram



Micro operations and control signals - PC group

Micro operation		PWu	PWc	Psrc
PC = PC + 4	PCinc	1	X	1
if (A == B) PC = Res	branch	0	1	0
PC=PC[31-28] s2(IR[25-0])	jump	1	X	2
default	nop	0	0	X

$$PW = PWu + Z \cdot PWc$$

Micro operations and control signals - Mem group

Micro operation		MW	MR	IorD	IW	DW
IR = Mem[PC]	fetch	0	1	0	1	0
DR = Mem[Res]	m_rd	0	1	1	0	1
Mem[Res] = R	m_wr	1	0	1	0	0
default	nop	0	0	X	0	0

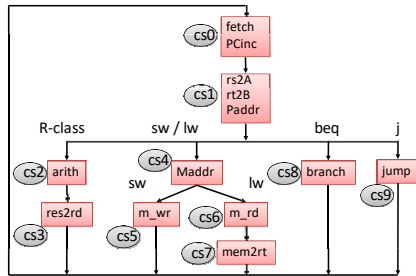
Micro operations and control signals - RF group

Micro operation		RW	Rdst	M2R	AW	BW
A = RF[IR[25-21]]	rs2A	0	X	X	1	0
B = RF[IR[20-16]]	rt2B	1	X	X	0	1
RF[IR[15-11]] = Res	res2rd	2	1	0	0	0
RF[IR[20-16]] = DR	mem2rt	1	0	1	0	0
default	nop	0	X	X	0	0

Micro operations and control signals - ALU group

Micro operation		bpc	Asrc1	Asrc2	ReW
PC = PC + 4	PCinc	0	0	1	0
Res = A op B	arith	2	1	0	1
Res = A + sx(IR[15-0])	Maddr	0	1	2	1
Res = PC + s2(sx(IR[15-11]))	Paddr	0	0	3	1
if (A == B) PC = Res	branch	1	1	0	0
default	nop	X	X	X	0

Control states and micro operations



Control states and signal values

	PCgrp	Mem grp	RF grp	ALU grp
CS0	PCinc	Fetch	nop	PCinc
CS1	nop	nop	rs2A_rt2B	Paddr
CS2	nop	nop	nop	arith
CS3	nop	nop	res2rd	nop
CS4	nop	nop	nop	maddr
CS5	nop	m_wr	nop	nop
CS6	nop	m_rd	nop	nop
CS7	nop	nop	mem2rt	nop
CS8	branch	nop	nop	branch
CS9	jump	nop	nop	nop

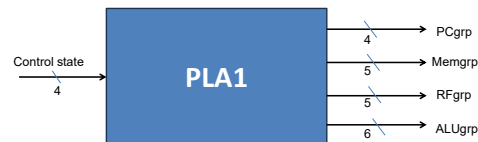
Control states and signal values

	PCgrp	Mem grp	RF grp	ALU grp
CS0	1X01	01010	0XX00	000010
CS1	00XX	00X00	0XX11	000111
CS2	00XX	00X00	0XX00	101001
CS3	00XX	00X00	11000	XXXXX0
CS4	00XX	00X00	0XX00	001101
CS5	00XX	10100	0XX00	XXXXX0
CS6	00XX	01101	0XX00	XXXXX0
CS7	00XX	00X00	10100	XXXXX0
CS8	0100	00X00	0XX00	011000
CS9	1X10	00X00	0XX00	XXXXX0

Control states and signal values

State	PCgrp	Mem grp	RF grp	ALU grp
0000	1X01	01010	0XX00	000010
0001	00XX	00X00	0XX11	000111
0010	00XX	00X00	0XX00	101001
0011	00XX	00X00	11000	XXXXX0
0100	00XX	00X00	0XX00	001101
0101	00XX	10100	0XX00	XXXXX0
0110	00XX	01101	0XX00	XXXXX0
0111	00XX	00X00	10100	XXXXX0
1000	0100	00X00	0XX00	011000
1001	1X10	00X00	0XX00	XXXXX0

PLA to generate control signals



Control state transitions

	R-class	sw	lw	beq	j
cs0	cs1	cs1	cs1	cs1	cs1
cs1	cs2	cs4	cs4	cs8	cs9
cs2	cs3	X	X	X	X
cs3	cs0	X	X	X	X
cs4	X	cs5	cs6	X	X
cs5	X	cs0	X	X	X
cs6	X	X	cs7	X	X
cs7	X	X	cs0	X	X
cs8	X	X	X	cs0	X
cs9	X	X	X	X	cs0

Control state transitions

State	R-class	SW	Lw	beq	j
0000	0001	0001	0001	0001	0001
0001	0010	0100	0100	1000	1001
0010	0011	XXXX	XXXX	XXXX	XXXX
0011	0000	XXXX	XXXX	XXXX	XXXX
0100	XXXX	0101	0110	XXXX	XXXX
0101	XXXX	0000	XXXX	XXXX	XXXX
0110	XXXX	XXXX	0111	XXXX	XXXX
0111	XXXX	XXXX	0000	XXXX	XXXX
1000	XXXX	XXXX	XXXX	0000	XXXX
1001	XXXX	XXXX	XXXX	XXXX	0000

Control state transitions

Current state	opcode					
	000000	101011	100011	000100	000010	
0000	0001	0001	0001	0001	0001	
0001	0010	0100	0100	1000	1001	
0010	0011	XXXX	XXXX	XXXX	XXXX	
0011	0000	XXXX	XXXX	XXXX	XXXX	
0100	XXXX	0101	0110	XXXX	XXXX	
0101	XXXX	0000	XXXX	XXXX	XXXX	
0110	XXXX	XXXX	0111	XXXX	XXXX	
0111	XXXX	XXXX	0000	XXXX	XXXX	
1000	XXXX	XXXX	XXXX	0000	XXXX	
1001	XXXX	XXXX	XXXX	XXXX	0000	

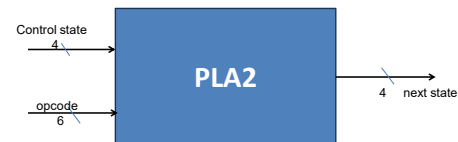
Control states transition-another way

CS0	X	CS1
CS1	R-class	CS2
CS1	Sw/lw	CS4
CS1	beq	CS8
CS1	j	CS9
CS2	X	CS3
CS3	X	CS0
CS4	sw	CS5
CS4	lw	CS6
CS5	X	CS0
CS6	X	CS7
CS7	X	CS0
CS8	X	CS0
CS9	X	CS0

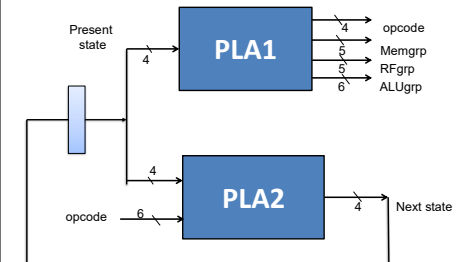
Control states transition-another way

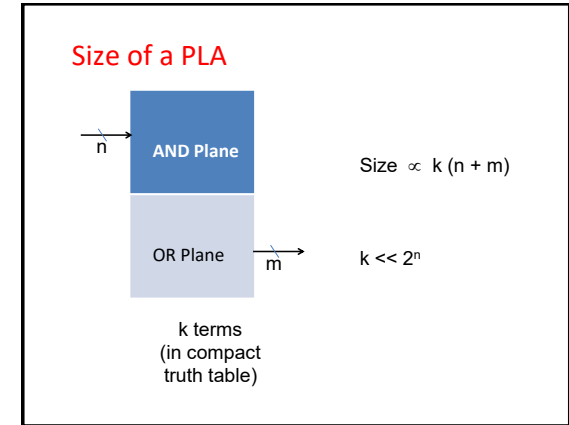
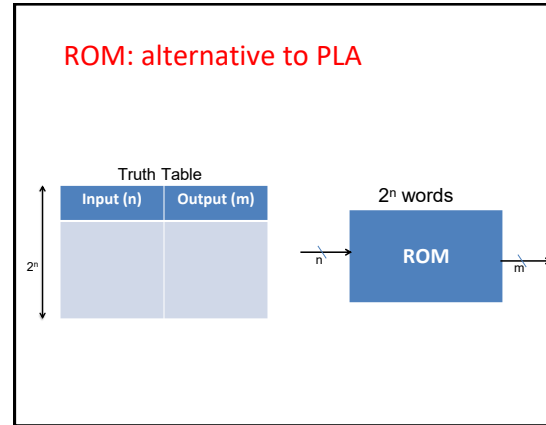
0000	xxxxxx	0001
0001	000000	0010
0001	10X011	0100
0001	000100	1000
0001	000010	1001
0010	xxxxxxx	0011
0011	xxxxxxx	0000
0100	101011	0101
0100	100011	0110
0101	xxxxxxx	0000
0110	xxxxxxx	0111
0111	xxxxxxx	0000
1000	xxxxxxx	0000
1001	xxxxxxx	0000

PLA to determine next state



Controller design with PLAs





0000	xxxxxxx	0001
0001	000000	0010
0001	10X011	0100
0001	000100	1000
0001	000010	1001
0010	XXXXXXX	0011
0011	XXXXXXX	0000
0100	101011	0101
0100	100011	0110
0101	XXXXXXX	0000
0110	XXXXXXX	0111
0111	XXXXXXX	0000
1000	XXXXXXX	0000
1001	XXXXXXX	0000

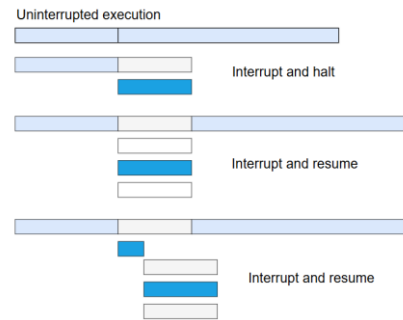
- Mechanism to alter the control flow
- Event identification and response to it
- Halt or resume original control flow

Terminology

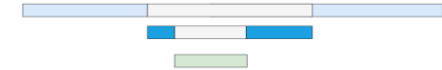
Exceptions vs. Interrupts

- Use both the terms interchangeably
- **MIPS**: exception - internal as well as external, interrupt - external
- **Intel x86**: interrupt - internal as well as external
- **PowerPC**: exception - unusual event, interrupt - change in flow of control

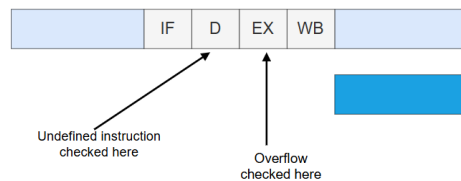
How exceptions are handled



Nested interrupts



Checking for exceptions (MIPS)



Changes required in datapath

Exception program counter

- Saves PC of the instruction causing exception
- For resuming interrupted program

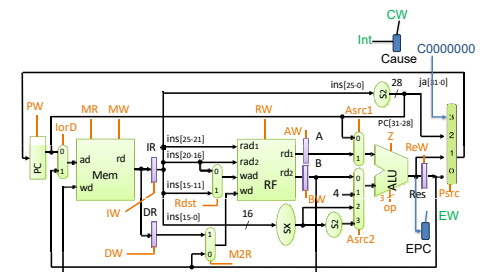
Cause register

- Records the cause of the exception
- 0: undefined instruction, 1: arithmetic overflow

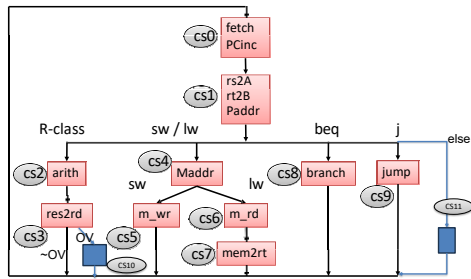
Address of the exception handler

- C000 0000 in our case
- Multiple addresses for vectored interrupts

Control signals in multi-cycle DP



Control states and micro operations



New control states

